

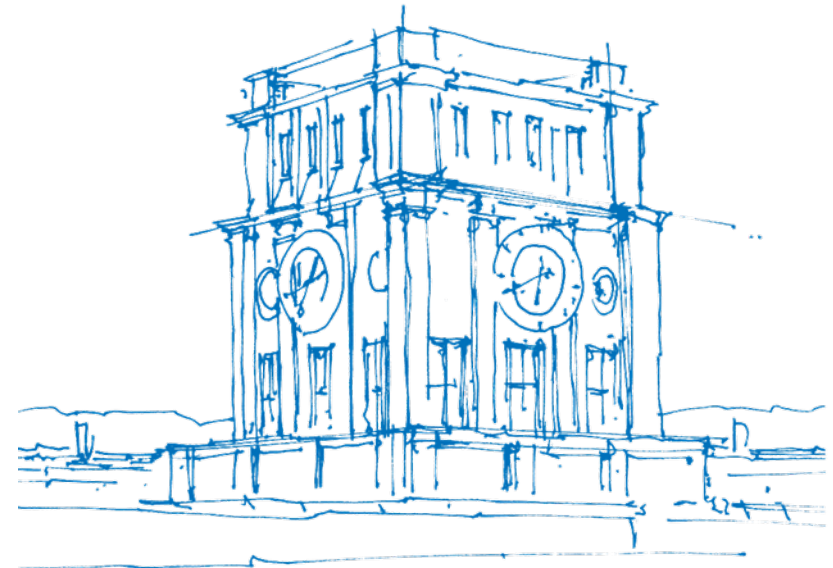
Formal Verification of Earley's Parsing Algorithm, Including Running Time Analysis

Alexander Haberl

Technische Universität München

TUM School of Computation, Information and Technology - Informatics

Munich, 06. May 2026



TUM Uhrenturm

Basic Notation

Nts: A, B, C Tms: a, b, c Sententials: α, β

Derivations: $P \vdash \alpha \Rightarrow^* \beta$

Productions: $A \rightarrow \alpha$

Fixed constants:

- input word $w = w_0 \dots w_{n-1}$
- start Symbol: S
- set of Productions: P

Notation is mostly informal and lemmas and theorems are sometimes simplified by dropping assumptions about invariants or well-formedness, we drop explicit conversion from lists to sets

Earley Recognizer

Compute item sets (bins) $\mathcal{S}$, Items consisting of a dotted production and a "*from*" index indicating a previous bin $(A \rightarrow \alpha \bullet \beta, \text{from})$.

Specification: $(A \rightarrow \alpha \bullet \beta, k) \in \mathcal{S}_i \implies P \vdash \alpha \Rightarrow^* w_k \dots w_{i-1}$

$$\begin{array}{ll} \text{INIT: } \frac{S \rightarrow \alpha \in P}{(S \rightarrow \bullet \alpha, 0) \in \mathcal{S}_0} & \text{PREDICT: } \frac{(A \rightarrow \alpha \bullet B \gamma, k) \in \mathcal{S}_i \quad B \rightarrow \beta \in P}{(B \rightarrow \bullet \beta, i) \in \mathcal{S}_i} \end{array}$$

$$\text{COMPLETE: } \frac{(A \rightarrow \alpha \bullet, k) \in \mathcal{S}_i \quad (B \rightarrow \beta \bullet A \gamma, l) \in \mathcal{S}_k}{(B \rightarrow \beta A \bullet \gamma, l) \in \mathcal{S}_i}$$

$$\text{SCAN: } \frac{(A \rightarrow \alpha \bullet b \beta, l) \in \mathcal{S}_i \quad w_i = b}{(A \rightarrow \alpha b \bullet \beta, l) \in \mathcal{S}_{(i+1)}}$$

Prior Work

Prior Work

Purely inductive definition of Earley recognizer for correctness proofs.

Definition

$$accepted = (S \rightarrow \alpha\bullet, \theta) \in \mathcal{S}_{|w|}$$

Lemma

$$accepted \iff P \vdash S \Rightarrow^* w$$

Recursive computation of bins:

Init and Scan compute starting set, which is then closed under Predict and Complete (using *Close*)

Lemma

$$i \leq |w| \implies bins\ |w| \ !\ i = \mathcal{S}_i$$

Prior Work

Refinement of closure in *Close2*, an inductively defined one-pass workset algorithm.

Notation: B_s : list of bins, B : workset, C : result, (B', C') : next state

Definition

Close2: $B_s \vdash (B, C) \rightarrow (B', C')$

$x \in B \wedge \neg is_complete\ x \implies B_s \vdash (B, C) \rightarrow (B \cup \text{Predict } x\ |B_s| - (C \cup \{x\}), C \cup \{x\})$

$x \in B \wedge is_complete\ x \implies B_s \vdash (B, C) \rightarrow (B \cup \text{Complete } B_s\ x - (C \cup \{x\}), C \cup \{x\})$

Pick the result of the workset algorithm for a starting workset B

$close2\ B_s\ B = (SOME\ C.\ B_s \vdash (B, \emptyset) \rightarrow^* (\emptyset, C))$

Lemma

$wf_bins1\ B_s \wedge wf_bin1\ B\ |B_s| \implies close2\ B_s\ B = Close\ B_s\ B$

Own Work

List Recognizer

Towards executability we implement the Earley operations on Lists, using filtering and mapping.

- $Init_L$ and $Predict_L$ filter the productions for a specific Nt
- $Scan_L$ filters a bin for items with a Tm after the dot
- $Complete_L$ filters a previous bin for items with a specific Nt after the dot

We implement the closure, where each step emulates the *Close2* steps

Definition

$step_fun_L Bs (b \# bs, cs) =$

$(let\ nexts = if\ is_complete\ b\ then\ Complete_L\ Bs\ b\ else\ Predict_L\ b\ |Bs|$

$in\ (nexts \cup_L (b \# bs) -_L (\{b\} \cup_L cs), \{b\} \cup_L cs))$

$steps_L Bs (B, C) = (if\ B \neq []\ then\ steps_L Bs (step_fun_L Bs (B, C))\ else\ (B, C))$

List Recognizer

We prove termination of $steps_L$ and equivalence to $close2$

Lemma

$\exists C'. steps_L Bs (B, C) = ([], C')$

$steps_L Bs (B, C) = ([], C') \implies Bs \vdash (B, C) \rightarrow^* ([], C')$

$bins$ using this closure is fully executable.

The running time is dominated by the list difference operation which is executed each step.

Efficient Item List Data Structure

Earley proposes data structure using an array to achieve a linear difference operation.

We stay purely functional and do a parameterized running time analysis instead.

Partition bin on the "from" index of items -> each part has constant size:

$$\begin{aligned} &[(A \rightarrow a\bullet bc, 0), (A \rightarrow B\bullet A, 2), (B \rightarrow b\bullet, 1), (A \rightarrow \bullet abc, 2), (A \rightarrow a\bullet a, 0)] \\ &[[(A \rightarrow a\bullet bc, 0), (A \rightarrow a\bullet a, 0)], \\ &\quad [(B \rightarrow b\bullet, 1)], \\ &\quad [(A \rightarrow B\bullet A, 2), (A \rightarrow \bullet abc, 2)]] \end{aligned}$$

Efficient Item List Data Structure

Definition

datatype efficientItemList = IL bin (parts list)

Invariant to ensure validity of the data structure (*IL as fs*):

$$x \in as \longleftrightarrow x \in fs \text{ ! (from } x)$$

Definition

diff_{LIL} k [] il = empty_{IL} k

diff_{LIL} k (a # as) il = (if isin_{IL} il a then diff_{LIL} k as il else a #_{IL} diff_{LIL} k as il)

Theorem

$$i \leq k \wedge k \leq |w| \implies bins_{IL} k \text{ ! } i = \mathcal{S}_i$$

$$accepted_{IL} \longleftrightarrow P \vdash S \Rightarrow^* w$$

Running Time Analysis

We perform a running time analysis with a parameter for the random access time of a part $T_{nth_{IL}}$, with the following assumptions:

Definition

$T_{nth_{IL}} :: nat \Rightarrow nat$

- $T_{nth} \text{ as } k \leq T_{nth_{IL}} k$
- $T_{list_update} \text{ as } k \ a \leq T_{nth_{IL}} k$
- $mono\ T_{nth_{IL}}$

For lists: $T_{nth_{IL}}\ n = n$

For arrays: $T_{nth_{IL}}\ n = 1$

For trees: $T_{nth_{IL}}\ n = \log |w|$

Running Time Analysis

Lemma

$$T_{isin_{IL}} \text{ il } x \leq T_{nth_{IL}} (\text{from } x) + C$$

$$T_{diff_{LIL}} k \text{ as il } \leq |as| * (4 * T_{nth_{IL}} k + C) + k$$

$$T_{close2_{IL}} Bs \text{ il } \leq (C + C1 * T_{nth_{IL}} |Bs|) * |Bs|^2 + O(|Bs|)$$

Theorem

$$T_{accepted_{IL}} P S w \leq (C1' + C2 * T_{nth_{IL}} |w|) * |w|^3$$

$$C1' = 30 * (L + 1)^3 * (K + 1)^3 \quad C2 = 17 * (L + 1)^2 * (K + 1)^2$$

Parse Tree Basics

Definition

***datatype** parseTree = Rule Nt (parseTree list) | Sym Tm*

root = Nt or Tm at the root of the tree

fringe = concatenation of Tms in the Sym leafs

valid_parse_tree P w A t = *t* is a valid parse tree for $P \vdash A \Rightarrow^* w$

Augment to a Parser

We save parse items that are an Earley item and parse trees. *parseItem* :: *item* $\times$ *tree list*

Specification:

$$((A \rightarrow BC\bullet Da, k), [t1, t2]) \in \mathcal{S}_i \implies$$

$$P \vdash BC \rightarrow^* w_k \dots w_{i-1} \wedge \text{root } t1 = B \wedge \text{root } t2 = C \wedge \text{fringe } t1 @ \text{fringe } t2 = w_k \dots w_{i-1}$$

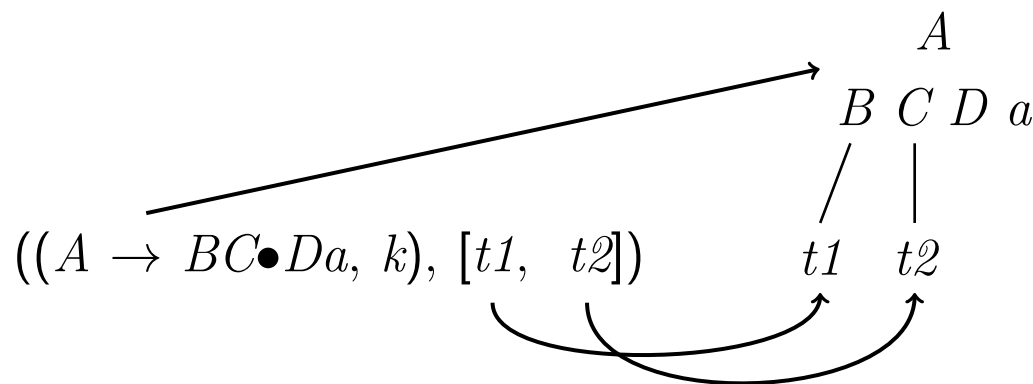


Figure: Visualization how the production and parse tree list represent one possibly incomplete parse tree.

Augment to a Parser

We have to augment the recognizer operations, specifically Complete and Scan to add trees:

$$\begin{array}{ll}
 \text{INIT: } \frac{S \rightarrow \alpha \in P}{((S \rightarrow \bullet\alpha, \theta), []) \in \mathcal{S}_0} & \text{PREDICT: } \frac{((A \rightarrow \alpha\bullet B\gamma, k), ts) \in \mathcal{S}_i \quad B \rightarrow \beta \in P}{((B \rightarrow \bullet\beta, i), []) \in \mathcal{S}_i}
 \end{array}$$

$$\text{COMPLETE: } \frac{((A \rightarrow \alpha\bullet, k), ts1) \in \mathcal{S}_i \quad ((B \rightarrow \beta\bullet A\gamma, l), ts2) \in \mathcal{S}_k}{((B \rightarrow \beta A\bullet\gamma, l), ts2 @ (\textit{Rule } A \ ts1)) \in \mathcal{S}_i}$$

$$\text{SCAN: } \frac{((A \rightarrow \alpha\bullet b\beta, l), ts) \in \mathcal{S}_i \quad w_i = b}{((A \rightarrow \alpha b\bullet\beta, l), ts @ (\textit{Sym } b)) \in \mathcal{S}_{(i+1)}}$$

Augment to a Parser

We augment the item list to also store parse items, split into items and trees.

Definition

$parseIL :: efficientItemList \times (parseTree\ list\ list)$

$set_{pIL} (il, ts) = set\ (zip\ (list\ il)\ ts)$

The parsing algorithm is the same as the recognizer, but uses the parse operations instead.

Top level function that returns a tree of a final item if it exists:

Definition

$get_parse_tree_w = if\ ((S \rightarrow \alpha\bullet, \theta), ts) \in \mathcal{S}_{|w|}\ then\ Some\ (Rule\ S\ ts)\ else\ None$

Augment to a Parser

Theorem

$$\exists t. \text{get_parse_tree_}w = \text{Some } t \iff P \vdash S \Rightarrow^* w$$

$$\text{get_parse_tree_}w = \text{Some } t \longrightarrow \text{valid_parse_tree } P \ w \ S \ t$$

Theorem

$$T_{\text{get_parse_tree_}w} \ P \ S \ w \leq (C + C1 * T_{nth_{LL}} \ |w|) * |w|^3 + O(|w|^2)$$